Wprowadzenie

Systemy Monolityczne

Cechy

- ściśle zintegrowane moduły
- wysoki koszt wprowadzania zmian
- “Dług technologiczny”
- ograniczenie skalowalności

Dane

- jedna relacyjna baza danyhc
- łatwa interpretacja i dostęp
- dojrzałe narzędzia do przetwarzania i analizy (RTL i BI)
- możliwość eksportu do hurtowni danych

SOA - *Service Oriented Architecture*

Usługa

- logiczna reprezentacja powtarzalnej czynności biznesowej o zdefiniowanym wyniku (np. dostarczenie prognozy pogody, sprawdzenie salda etc.)
- jest samodzielnym bytem
- może się składać z innych usług
- dla klienta stanowi “black box”

Cechy

- wyodrębnienie niezależnych usług
- koncentracja na zaprojektowaniu dobrego API
- przykrycie szczegółów implementacyjnych
- możliwość niezależnego skalowania
- wyniesienie procesów
- rozproszenie danych

Dane

- problem z tradycyjnym podejściem ETL
- enterprise Service Bus (ESB)
- możemy odpytywać cyklicznie API
- event-driven architecture (EDA) - model push
  - usługi publikują zdarzenia
  - Inne usługi zapisują się na nie

Mikrousługi

Cechy

- duża liczba wyspecjalizowanych usług wykonujących dobrze zdefiniowane funkcjonalności
- rezygnacja z ESB
- model aktorowy
- większa różnorodność technologii programistycznych i bazodanowych
- odporność na awarie

Dane

- bazy NoSQL

- pub-sub model
- problemy z transakcyjnością
- CAP theorem
- mechanizmy kompensacyjne

CAP theorem

W danej chwili czasu możemy zachować co najwyżej dwie z trzech cech systemu:
- spójność (Consistency)
- dostępność (Availability)
- rozproszenie (Partitioning tolerance)

ACID vs BASE
- ACID - Atomicity, Consistency, Independence, Durability
- BASE - Basically Available, Soft state, Eventually consistent

Agenci AI

Agenty AI to moduły programowe zasilane komponentami AI zdolne do autonomicznej realizacji zadań, podejmowania decyzji, uczenia i interakcji ze środowiskiem w celu osiągnięcia założonych celów. Rozwój technologii AI, w szczególności GenAI, pozwala na aktywne wspieranie ludzi w realizacji złożonych procesów biznesowych jako wirtualni współpracownicy.

Zastosowania
- asystenci programowania (*co-pilot*)
- edukacja
- robotyka
- gry wideo

Rodzaje baz danych NoSQL
- klucz-wartość
- szerokie kolumny
- dokumenty
- grafy
- XML
- RDF
- silniki wyszukiwania
- content stores

Chmura

Chmura dostarcza zasoby obliczeniowe i możliwośc składowania danych w modelu self-service, z możliwością automatycznego skalowania i monitorowania.

Modele chmury
- software as a Service (SaaS) - usługa dostarczająca konretną funkcjonalność (np. email, ERP etc.)
- platform as a Service (PaaS) - środowisko do tworzenia usług
- infrastructure as a Service (IaaS) - wirtualne maszyny, storage etc.
- database as a Service (Daas)
- testing as a Service (TaaS)

Rodzaje analityki w chmurze
- SaaS - gotowe do zastosowania znane algorytmy data mining, knowledge discovery, machine learning

- PaaS - dostarczenie środowiska do budowy własnych narzędzi i algorytmów analitycznych przykrywając infrastrukturę
- IaaS - algorytmy dostępne w postaci usług o dobrze zdefiniowanym API

IOT

- Internet of Things (IoT) - internet fizycznych obiektów obdażonych mocą obliczeniową i połączonych za pomocą Internetu
- Edge Computing - Przenoszenie obliczeń do brzegu sieci, jak najbliżej ucządzeń korzystających z tych obliczeń
- Cyber Physical Systems (CPS) - połączenie obliczeń z procesami fizycznymi

Big Data

- systemy, w których dane nie dają się przetworzyć tradycyjnymi metodami
- przetwarzane zarówno w czasie rzeczywistym (np. transakcje), jak i w szerszym oknie (próba odnalezienia trendów, typy klientów itp.)
- nazwa popularyzowana od końca lat 90. (np. John Mashey, 1998)
- przyczyny rozwoju:
  - potrzeby nauki i biznesu
  - rozwój chmury obliczeniowej
  - rozwój internetu rzeczy
  - tanie nośniki danych
- przykładowe dziedziny:
  - służba zdrowia
  - marketing
  - astronomia

Cechy 4V

- Volume - rozmiar danych
- Variety - różnorodność danych
- Velocity - prędkość napływu danych
- Veracity - nierówna jakość danych

Źródła danych Big Data

Internet

- Internet jest jednym z największych zbiorów danych tworzonych przez ludzkość
- dane mają różny poziom ustrukturyzowania, jakości i spójności
- zastosowania:
  - web analytics
  - web mining
  - opinion mining

Dane naukowe

- biomedyczne
  - obrazowanie medyczne generuje bardzo duże wolumeny danych
  - sekwencjonowanie genomu to kolejne źródło dużych zbiorów danych
- astronomiczne
  - teleskopy i symulacje naukowe generują dziennie dziesiątki lub setki TB
- fizyczne
  - eksperymenty takie jak LHC generują setki milionów zdarzeń na sekundę

Obliczenia rozproszone

Map-reduce

- map reduce - narzędzie kojarzone z Big Data, do przetwarzania danych na dużych klastrach obliczeniowych
- algorytm:
  - map - generujemy pary klucz-wartość
  - shuffle - przesyła pary o tym samym kluczu do jednego węzła
  - reduce - przetwarza otrzymane pary
- często przepisujemy zapytania SQL na map-reduce
- dzięki takiemu modelowi łatwo zaimplementować:
  - algorytmy zliczania słów/znaków w wielkich zbiorach tekstowych
  - język SQL
  - raporty podsumowujące transakcje w sklepie internetowym
- wady:
  - konieczność zapisu wyników pośrednich na dysk
  - sztywny format obliczeń w formie sekwencji par map-reduce

Bulk Synchronous Parallel

- abstrakcyjna idea, niepowiązana z konkretnym narzędziem
- cel - stworzenie systemu obliczeń:
  - równoległego
  - odpornego na awarie
  - wolnego od deadlocków
  - łatwego do prowadzenia wielkoskalowych obliczeń
- **supersteps**
  - w ramach jednego superkroku wykonywane jest mnóstwo równoległych obliczeń/procesów
  - gdy się wymienią wynikami to dopiero inicjowany jest kolejny superkrok
  - wcześniej bariera zatrzymuje procesy
- koszt kroku superstep:
  - $t_{\text{step}} = \max(w_i) + mg \max\{h_i\} + I$
  - w_i - czas pracy dla procesora i
  - h_i - liczba przychodzących i wychodzących wiadomości dla procesora i
  - m - rozmiar wiadomości
    - g - przepustowość danych do procesora
    - I - opóźnienie sieci

Pregel

- implementacja BSP od Googla
- wersja BSP dostosowana do przetewarzania w grafach
- obliczenia modelowane jako DAG (**Directed Acyclic Graph**)
- funkcje obliczane w węzłach
- cykle obliczeniowe - “supersteps”
- możliwa zmiana topologii grafu w trakcie obliczeń
- w każdej iteracji węzeł może:
  - odebrać wiadomość wysłaną w poprzedniej iteracji
  - wysłać wiadomości do innych węzłów
  - zmienić swój stan
  - zmodyfikować topologię grafu

DRYAD

- BSP od Microsofta
- proces obliczeniowy jako DAG
- struktura grafu może ewoluować w trakcie obliczeń (zmieniać się w trakcie liczenia)

GraphLab

- model obliczeń dedykowany problemom MLDM (*Machine Learning, Data Mining*)
- obliczenia składają się z grafu oraz tablicy klucz-wartość
- wyniki częściowe nie muszą być zapisywane na dysk
- dwa główne kroki:
  - *update* - obliczenia lokalne dla węzła i jego otoczenia
  - *sync* - agregacja wyników częściowych

Model Aktorowy

- cechy aktora
  - trwały
  - ma stan wewnętrzny
  - asynchroniczny
- ważna cecha układu - deterministyczny - aktor jest pasywny do czasu nadejścia wiadomości
- gdy przyjdzie wiadomość aktor może
  - wysłać skończoną liczbę wiadomości do innych aktorów
  - stworzyć skończoną liczbę innych aktorów
  - wybrać wywoływaną procedurę po otrzymaniu kolejnej wiadomości
- po awarii nadzorca może:
  - wznowić aktora zachowując stan
  - zrestartować go czyszcząc stan
  - zatrzymać jego działanie
  - eskalować błąd wyżej
- cechy jako model Big Data
  - zalety
    - niskopoziomowe zrównoleglenie - systemy z tysiącami aktorów
    - odporność na awarię - pojedynczy aktor nie blokuje całego systemu
    - dobrze nadaje się do modelowania przepływów i transformacji danych
    - elastyczne skalowanie
  - wady
    - szybko rośnie ilość komunikatów
    - trudne debugowanie
    - nieograniczone kolejki wiadomości mogą przeciążyć system
- przykłady - Akka, Erlang, Pulsar

Information Flow Processing (IFP)

DSMS

- *Data Stream Management Systems*
- porównanie z DBMS:
  - operują na strumieniach zamiast trwałych relacji
  - używają zapytań ciągłych zamiast jednorazowych
  - zakładają sekwencyjny dostęp i wysoką dynamikę napływu danych
  - często mają wymagania *real-time*

DSMS – Data Stream Management Systems

DBMS	DSMS
Persistent data (relations)	volatile data streams
Random access	Sequential access
One-time queries	Continuous queries
(theoretically) unlimited secondary storage	limited main memory
Only the current state is relevant	Consideration of the order of the input
relatively low update rate	potentially extremely high update rate
Little or no time requirements	Real-time requirements
Assumes exact data	Assumes outdated/inaccurate data
Plannable query processing	Variable data arrival and data characteristics

Table: DBMS vs DSMS

Active Databases

- posiada funkcjonalności typowej bazy danych
- identyfikuje nadejście nowego zdarzenia
- sprawdza warunki i uruchamia akcję
- posiada silnik uruchamiania reguł

CEP

- *Complex Event Processing*
- pozwala na przetwarzanie zdarzeń i reagowanie na nie
- odnajduje wzorce w sekwencjach zdarzeń
- możliwości filtrowania, agregacji
- śledzenie zależności między zdarzeniami (przyczynowe, czasowe itp.)

Model publish-subscribe

- model wymiany informacji między **luźno** powiązanymi usługami
- **brokerzy** pośredniczą w przekazywaniu wiadomości
- oddziela operację generowania informacji (wiadomości, zdarzenia, porcji danych itp.) od jej konsumpcji
- pozwala na przesyłanie danych w modelu wiele do wielu
- asynchroniczne działanie
- może stanowić bufor zwiększający odporność systemu na awarie komponentów
- **offset** - punkt ostatniego odczytu dla grupy konsumentów
  - przydatny do cofania się po awarii
  - przechowywany jest zwykle przez zarządcę modelu publish-subscribe
- przykład implementacji: Kafka
- rodzaje strumieni
  - *at-most once* - każda wiadomość zostanie dostarczona **co najwyżej raz** (jeżeli nie zostanie przetworzona, zostaje utracona)
  - *at-least once* - każda wiadomość zostanie dostarczona **co najmniej raz** (mogą pojawić się duplikaty gdy np. odbiorca nie potwierdzi odbioru)
  - *exactly once* - wiadomość zostanie dostarczona **dokładnie raz** (trudne do osiągnięcia w praktyce)

Inne modele i techniki

All-Pairs

- wzorzec obliczeń polegający na porównywaniu wszystkich elementów ze wszystkimi
- naiwnie ma bardzo wysoką złożoność i słabą skalowalność na dużych danych

Roaring Bitmaps

- skompresowana reprezentacja zbiorów liczb całkowitych
- pozwala wydajnie wykonywać operacje takie jak przecięcie, suma i filtrowanie
- przydatna przy indeksowaniu i analityce na dużych zbiorach

GPGPU

- wykorzystanie kart graficznych do obliczeń ogólnego przeznaczenia
- dobre dla operacji wykonywanych równolegle na dużych zbiorach danych
- typowe zastosowania:
  - przetwarzanie macierzy
  - uczenie maszynowe
  - symulacje

Graph Computing Systems

- systemy przetwarzania grafowego wykonują iteracyjne obliczenia na całym grafie
- bazy grafowe wspierają analizę online całego grafu lub jego fragmentów
- przykłady:
  - Apache Giraph
  - Neo4j

Przetwarzanie strumieni danych

- rekordy są przetwarzane pojedynczo lub w małych porcjach
- obliczenia definiuje się zwykle jako topologie DAG
- dla operacji stanowych trzeba śledzić stan obliczeń

Operacje

- bezstanowe
  - filtrowanie
  - mapowanie
- stanowe
  - agregacje
  - okna czasowe
- terminalne
  - zapis do bazy, pliku lub systemu zewnętrznego

Mikrobatch

- rekordy są przetwarzane małymi porcjami
- poprawia wydajność przy zachowaniu opóźnień zbliżonych do *real-time*

Unbounded table

- strumień można traktować jak nieograniczoną tabelę
- kolejne rekordy pojawiają się w niej na bieżąco

Architektury

Przetwarzanie wsadowe vs strumieniowe

- wsadowe przetwarzają w jednym przebiegu dużą ilość historycznych danych
- strumieniowe przetwarzają po jednym rekordzie na raz

	Wsadowe	**Strumieniowe**
historia	długa	wolna
prędkość	wolna	szybka
częstotliwość	niska	duża

Wyzwania w zarządzaniu architekturami Big Data

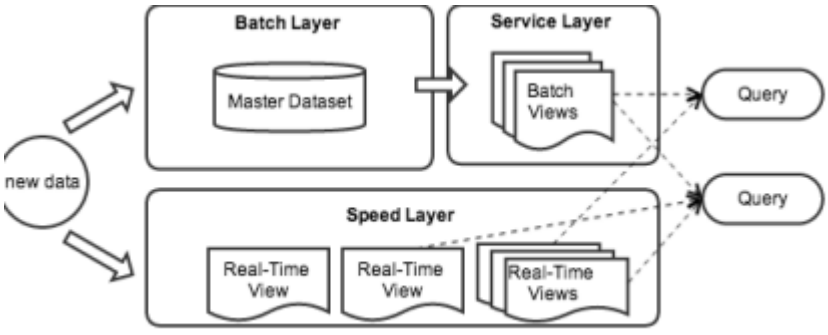
- podział komponentów architektury pomiędzy specjalizowane zespoły powoduje powstawanie silosów
- zmiany są kosztowne i rozległe
- wiele interakcji i interfejsów
- prawo Conwaya - projekt systemu IT odzwierciedla strukturę organizacyjną firmy

Architektura Data Lake



- cechy:
  - efektywna kosztowo
  - elastyczność formatów
  - opóźnienia
  - heterogeniczność źródeł danych

Architektura Lambda



- warstwy:
  - *batch-layer* - przetwarzanie danych historycznych
  - *speed-layer* - przetwarzanie w czasie rzeczywistym
  - *serving-layer* - udostępnianie widoków z przetworzonymi danymi

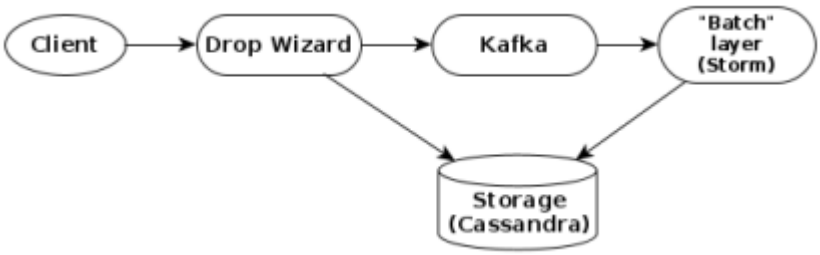
- w każdej kolejnej iteracji dane przetworzone tylko w strumieniu przetwarzane są wsadowo (w strumieniu są tylko najnowsze dane)
- zalety:
  - łączenie widokow batch i real-time
  - odporna na awarie i błędy
  - inkrementalna obsługa przyrostów danych
  - możliwość wprowadzenia usprawnień kodu które zostaną uwzględnione w kolejnym przeliczeniu dla całych danych historycznych
- wady:
  - konieczność utrzymywania dwóch strumieni (droga infrastruktura)
  - potencjalna duplikacja kodu

Przykład zastosowania

- system rekomendacji:
  - warstwa batch analizuje historię zachowań użytkowników
  - warstwa speed reaguje na bieżącą sesję
  - wynik końcowy łączy oba widoki

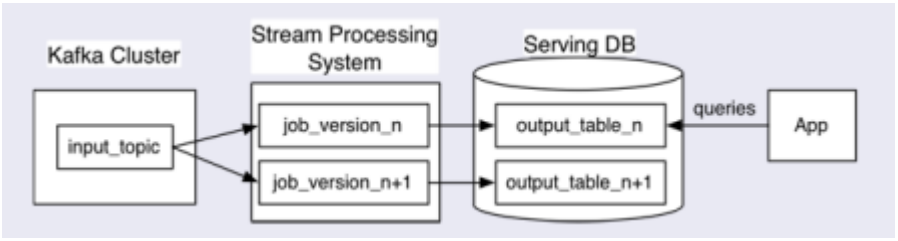
Architektura Delta

- odwrócona Lambda



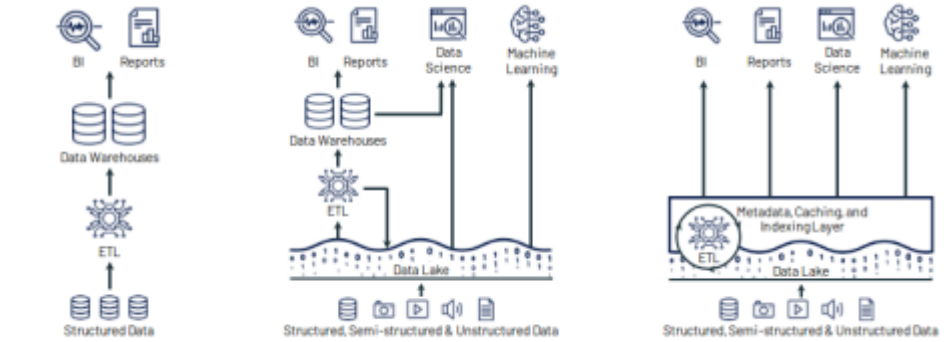
- odwraca role komponentów
- koncentruje się na przyrostach (deltach) danych

Architektura Kappa



- jedna ścieżka przetwarzania danych
- konieczność ponownego przepuszczenia danych historycznych przez algorytmy real-time
- zalety:
  - wprowadzenie nowego kodu do równoległego przetwarzania
  - możliwość przeliczenia całej dostępnej historii w modelu streaming
  - przesunięcie offsetu na początek dostępnego strumienia danych
  - przełączanie usług na nowe widoki po nadgonieniu przetwarzania przez nową wersję

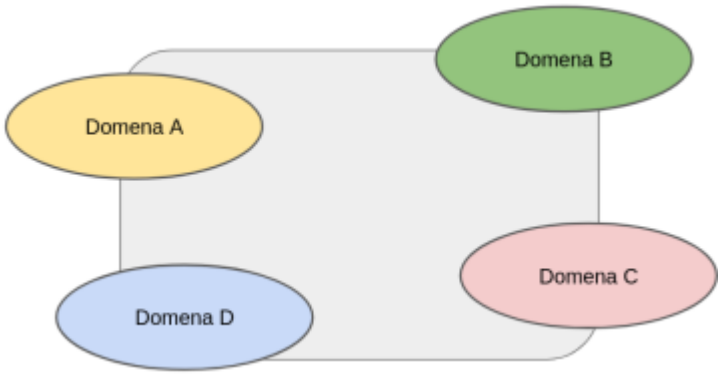
Architektura Lakehouse



(a) First-generation platforms. (b) Current two-tier architectures. (c) Lakehouse platforms.

- rozwinięcie Data Lake
- dodaje spójność, schemat, elementy transakcyjności
- dodatkowy narzut na scentralizowane zarządzanie danymi
- tworzy silosy danych

Data Mesh



- zdecentralizowana odpowiedzialność za dane z różnych domen
- dane jako produkt dostarczany przez różne zespoły
- skalowalność, niezależność, *Domain-Driven Design*
- przykład - e-commerce:
  - domeny:
    - klient (konta, autentykacja itp.)
    - zamówienia (koszyk, płatność itp.)
    - produkt (katalog, stany magazynowe itp.)
    - analityka (segmentacja klientów, rekomendacje itp.)
  - każdy zespół odpowiedzialny za:
    - jakość danych
    - schemat
    - dokumentację
    - bezpieczeństwo
    - transformacje

Spójne przetwarzanie zdarzeń

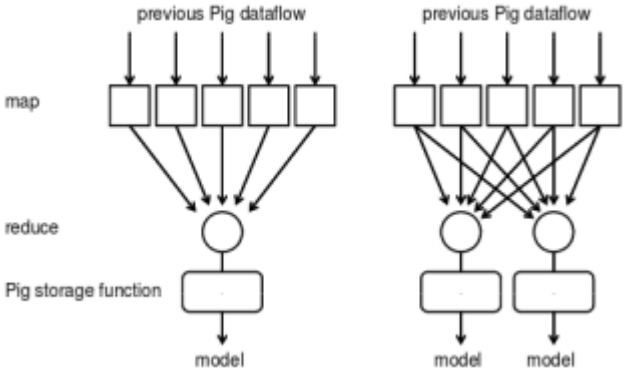
- celem może być uzyskanie semantyki *exactly-once*
- często wyniki publikowane są w stałych interwałach, np. co godzinę
- ważne pojęcia:

- *offset* - pozycja w strumieniu
- czas zdarzenia i czas przetwarzania

SQL na danych rozproszonych

- Hive / Tez
  - wykonywanie zapytań SQL w środowisku Hadoop
- SparkSQL
  - SQL nad danymi przetwarzanymi wsadowo i strumieniowo

Data mining w Twitter



- zastosowanie języka przepływu danych wysokiego poziomu Pig
- podział transformacji na re-używalne biblioteki
- wykorzystanie orkiestracji danych

Inne przykłady architektur analitycznych

Starfish

- optymalizacje dla środowiska Hadoop
- strojenie parametrów zadań i planu wykonania
- optymalizacja serializacji i współdzielenia danych

Kylin

- interfejs SQL i analityka OLAP nad Hadoop
- prekalkulacja agregatów dla szybkich zapytań analitycznych

MLBase

- przykład architektury do zastosowań uczenia maszynowego

Bazy danych NoSQL

Bazy key-value

- przechowują pary klucz-wartość
- nie wymagają predefiniowanej struktury
- optymalizacja na szybkość odpytywania pojedynczych rekordów po kluczu
- np. obsługa sesji

Bazy kolumnowe

- dane grupowane po kolumnach a nie wierszach:
  - baza wierszowa na dysku: `[1, Anna, 30, Warszawa] [2, Piotr, 25, Kraków]`
  - baza kolumnowa na dysku: `[1, 2] [Anna, Piotr] [30, 25] [Warszawa, Kraków]`

- luźny schemat (np. dodawanie nowych kolumn w locie)
- zoptymalizowane na przetwarzanie całych kolumn (np. w zapytaniach agregujących `sum`, `max`, `avg`)

Bazy dokumentowe
- pary klucz-wartość w formie dokumentów
- dokumenty grupowane w kolekcje
- duża elastyczność bez ścisłego formatu

Bazy grafowe
- dane w postaci wierzchołków i krawędzi
- dobrze odzwierciedlają relacje
- zoptymalizowane po przeszukiwanie grafu

(De)serializacja
- **serializacja** - proces konwersji struktur danych na strumień bitów w celu zapisu na dysk, przesłania do innych węzłów itp.
- **deserializacja** - proces odwrotny

Formaty tesktowe
- np. XML, CSV, JSON, YAML
- kodują każdą wartość danych jako znaki ASCII/UTF-8
  - np. liczba `30` to są 2 bajty `0x33` i `0x30`
- czytelne dla człowieka
- opcjonalny schemat
- wolne parsowanie

Formaty binarne
- np. protobuf, MessagePack
- zapis typów i wartości bezpośrednio jako bajty
  - np. liczba `30` jest zapisana bezpośrednio jako `0x1E`
- ekstremalnie szybkie parsowanie

Porównania praktyczne
- formaty tekstowe vs binarne:
  - tekstowe są bardziej czytelne dla człowieka
  - binarne są zwykle szybsze i bardziej kompaktowe
- popularne formaty w ekosystemie Hadoop:
  - Avro - format wierszowy, dobry do wymiany danych
  - Parquet - format kolumnowy, dobry do analityki

Orkiestracja danych
- zbiór technologii które wprowadzają abstrakcję danych ponad warstwę ich przechowywania
- zalety:
  - ograniczenie powtarzalnej pracy
  - skracanie czasu na tworzenie nowych usług
  - ułatwienie zmiany technologii
  - płynna migracja do chmury

Wizualizacja danych

Czym jest wizualizacja danych

- wizualizacja to nie tylko „ładne obrazki”, ale sposób reprezentacji danych wspierający myślenie i wnioskowanie
- jej celem może być:
  - eksploracja danych
  - wyjaśnianie wyników
  - perswazja
- wizualizacja angażuje bardzo silne mechanizmy percepcyjne człowieka
- dobrze zaprojektowana pomaga dostrzec wzorce, zależności i odstające obserwacje

Dlaczego warto wizualizować

- klasyczne statystyki opisowe nie zawsze ujawniają strukturę danych
- przykładem jest kwartet Anscombe’a:
  - zbiory mogą mieć bardzo podobne statystyki
  - ale po narysowaniu wykresów okazują się zupełnie różne
- mózg człowieka bywa bardzo skuteczny w rozpoznawaniu:
  - kształtu
  - domknięcia
  - długości
  - zakrzywienia
  - gęstości
  - koloru
  - wielkości

Obszary wizualizacji

- wizualizacja danych nie jest dziedziną jednorodną
- główne zastosowania:
  - *data exploration* - szukanie wzorców i hipotez
  - *data explanation* - komunikowanie wyników odbiorcy
  - perswazja
  - infografika
- pokrewne rozróżnienia:
  - EDA - *Exploratory Data Analysis* (John Tukey)
  - *SciViz* - wizualizacja naukowa
  - *InfoViz* - wizualizacja informacji

Exploration vs Explanation

Data Exploration

- służy analitykowi lub badaczowi
- wspiera odkrywanie nieoczywistych zależności
- często jest interaktywna
- może wykorzystywać wiele widoków danych i szybkie filtrowanie

Data Explanation

- służy komunikowaniu już zrozumianych wniosków
- wizualizacja jest bardziej uporządkowana i celowa
- ważny jest kontekst, narracja i dobór najważniejszych elementów

Persuasion i deception

- wizualizacja może przekonywać odbiorcę do określonej interpretacji
- źle zaprojektowana albo intencjonalnie zmanipulowana może wprowadzać w błąd
- typowe ryzyka:
  - przesadne akcentowanie różnic
  - odwracanie przekazu
  - niewłaściwe skale
  - pomijanie istotnego kontekstu

Krótka historia wizualizacji

- wizualizacja danych jest dziedziną bardzo starą
- ważne przykłady historyczne:
  - William Playfair
    - rozwój wykresów liniowych i słupkowych
    - spopularyzowanie wykresu kołowego
  - Florence Nightingale
    - wykorzystanie wykresów do wnioskowania i komunikacji problemów zdrowotnych
  - John Snow
    - mapa przypadków cholery jako narzędzie analizy przyczyn epidemii
  - Charles Minard
    - słynna wizualizacja kampanii Napoleona jako przykład bardzo bogatej reprezentacji danych

Proces wizualizacji

- wizualizacja łączy:
  - obróbkę danych
  - mapowanie danych na reprezentację graficzną
  - wyświetlanie
  - postrzeganie
  - interpretację
  - zrozumienie
- projektant koduje informacje w formie wizualnej
- użytkownik dekoduje je przez percepcję i interpretację

Dane i ich interpretacja

- zanim powstanie wykres trzeba zrozumieć dane
- ważne pytania:
  - jakie są wymiary danych?
  - co jest obserwacją?
  - jaki jest typ danych?
  - jaka jest ich interpretacja konceptualna?
- ten sam zbiór wartości może mieć różne interpretacje
- przykład:
  - dane liczbowe mogą oznaczać temperaturę
  - wtedy można rozważać skalę nominalną, porządkową lub liczbową w zależności od pytania

Typy danych i zależności

- podstawowe typy danych:
  - nominalne
  - porządkowe
  - liczbowe

- dobór wizualizacji zależy od:
  - typu zmiennych
  - liczby wymiarów
  - zależności między zmiennymi
- klasyczny przykład Minarda pokazuje, że jedna wizualizacja może kodować jednocześnie:
  - czas
  - położenie
  - liczebność
  - kierunek marszu
  - temperaturę

Mapowanie danych na atrybuty graficzne

- według Jacques'a Bertina dane mapujemy na kanały wizualne
- najważniejsze kanały:
  - pozycja
  - wielkość
  - odcień
  - tekstura
  - kolor
  - kierunek
  - kształt
- nie każdy kanał jest równie dobry dla każdego typu danych
- zwykle:
  - pozycja jest bardzo silnym kanałem
  - wielkość dobrze oddaje porządek i liczby
  - kształt i kolor lepiej nadają się do rozróżniania kategorii

Kolor

- kolor opisuje się zwykle modelami RGB lub HSV
- dobór kolorów ma duży wpływ na czytelność wykresu
- pomocne narzędzia:
  - ColorBrewer
  - IWantHue
- problemy z kolorem:
  - różnice kulturowe w interpretacji barw
  - złudzenia optyczne
  - daltonizm

Wartości ciągłe

- kuszące jest użycie tęczowej palety kolorów
- zwykle jest to błąd, ponieważ:
  - ludzie grupują kolory w klasy
  - taka paleta bywa nieczytelna dla daltonistów
  - kolejność kolorów nie ma naturalnego znaczenia semantycznego
- lepiej stosować skale uporządkowane percepcyjnie

Efektywność kodowań wizualnych

- nie każde mapowanie danych jest równie skuteczne
- niektóre reprezentacje pozwalają szybciej i dokładniej porównywać wartości niż inne
- znaczenie mają m.in. badania Jocka Mackinlaya i Jeffreya Heera

- dobry projekt powinien minimalizować wysiłek interpretacyjny odbiorcy

Wybór formy wizualizacji

- dla tych samych danych można zbudować bardzo wiele poprawnych wykresów
- wybór rośnie jeszcze bardziej przy:
  - grupowaniu
  - filtrowaniu
  - transformacjach
  - interakcji
- sposoby pracy:
  - ręczne eksperymentowanie
  - wybór z katalogu znanych form
  - podejście półautomatyczne

Narzędzia do wizualizacji

- istnieje bardzo dużo narzędzi o różnym poziomie abstrakcji
- zgrubna hierarchia:
  - kreatory wykresów
    - Excel, Google Charts, Power BI, Tableau
  - gramatyki grafiki
    - ggplot2, D3, Vega
  - języki i biblioteki wizualizacji
    - matplotlib, Seaborn, Bokeh
  - uniwersalne API graficzne
    - OpenGL, Metal, WebGL
  - środowiska i silniki
    - Processing, Unity, Unreal Engine
- kompromisy:
  - łatwość użycia vs elastyczność
  - szybkość prototypowania vs otwartość rozwiązania

Grammar of Graphics

- gramatyki graficzne opisują reguły budowy wizualizacji z prostszych elementów
- popularny model to *Layered Grammar of Graphics* spopularyzowany przez Hadleya Wickhama
- główne składniki:
  - *data* - dane wejściowe
  - *aesthetics (aes)* - mapowanie danych na zmienne graficzne
  - *geom* - obiekty geometryczne reprezentujące dane
- dodatkowe elementy:
  - *scale* - skale i transformacje
  - *statistical* - operacje statystyczne
  - *facets* - podział na wiele małych wykresów
  - *coordinate system* - układ współrzędnych
  - *theme* - warstwa stylistyczna

Dane wielowymiarowe

- dla dwóch zmiennych naturalne jest mapowanie 2D, np. x i y
- dla większej liczby wymiarów można używać dodatkowych kanałów:
  - kolor
  - wielkość

    - przezroczystość
    - kształt
    - kierunek
- popularne techniki:
    - podwykresy (*small multiples*)
    - współrzędne równoległe (*parallel coordinates*)
    - animacja i interakcja

Wizualizacja danych wielkoskalowych

- przy bardzo dużych zbiorach danych pojawia się problem wydajności i czytelności
- dla *big tall data* stosuje się zwykle:
    - próbkowanie
    - agregację
    - modelowanie
- agregacja wymaga dobrych decyzji dotyczących:
    - siatki
    - poziomu szczegółowości
    - kodowania kolorów
- dla danych czasu rzeczywistego głównym celem jest minimalizacja opóźnień

Strategie dla real-time

- indeksowanie po stronie klienta
- *prefetching* i cache
- aproksymacja
- możliwe jest też modelowanie zachowania użytkownika w celu przewidywania kolejnych operacji

Dobre praktyki

- najpierw pokazuj dane, a nie dekoracje
- forma powinna wspierać treść
- projekt powinien być czytelny i adekwatny do celu
- interakcja ma pomagać, a nie ukrywać chaos projektu
- wizualizacja powinna wspierać zrozumienie, a nie tylko robić wrażenie

Czego unikać

- przeładowania ozdobnikami
- nieczytelnych skal kolorów
- zbyt wielu kanałów naraz
- nadużywania 3D
- manipulacji skalą lub zakresem osi
- wizualnych sztuczek prowadzących do błędnych wniosków

Zasada końcowa

- wizualizacja nie powinna kłamać
- nawet estetyczna grafika jest zła, jeśli zniekształca interpretację danych

Infrastructure as Code (IaC) w erze chmur

Definicja IaC

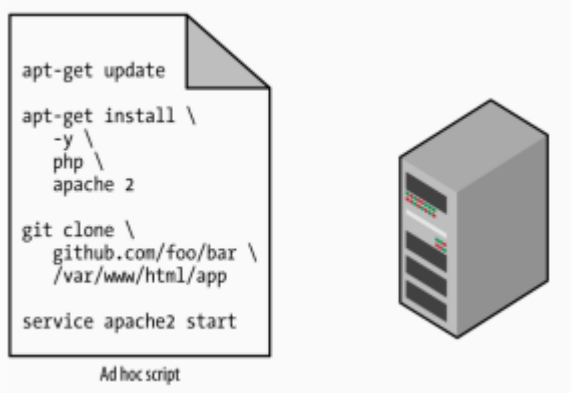
- pozwala na definiowanie, wdrażanie, modyfikacje oraz usuwanie infrastruktury poprzez pisanie i wykonanie kodu
- deklaratywny styl pisania kodu
- dobrze pasuje do zwirtualizowanego hardwaru
- reguły:
  - kontrola wersji - odtwarzanie zmian
  - przewidywalność - zawsze to samo środowisko
  - spójność - wiele instancji tego samego schematu są bardzo podobne do siebie
  - modułowość

Style IaC

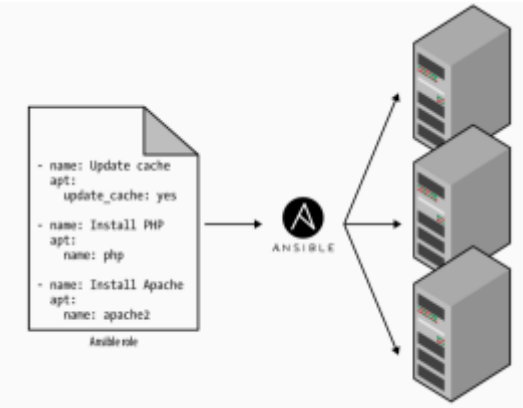
- *what* - podejście deklaratywne, opisujemy stan docelowy
- *how* - podejście imperatywne, opisujemy kroki wykonania
- provisioning:
  - tworzenie i modyfikacja zasobów przez API dostawców
- immutable infrastructure:
  - zamiast ręcznie zmieniać serwer, budujemy nowy obraz lub kontener
  - ogranicza problem *configuration drift*

Kategorie narzędzi IaC

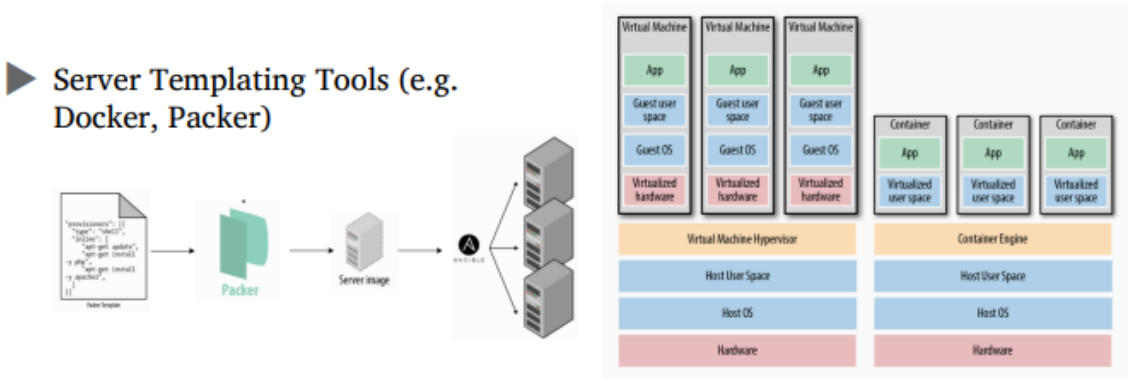
- skrypty ad-hoc:



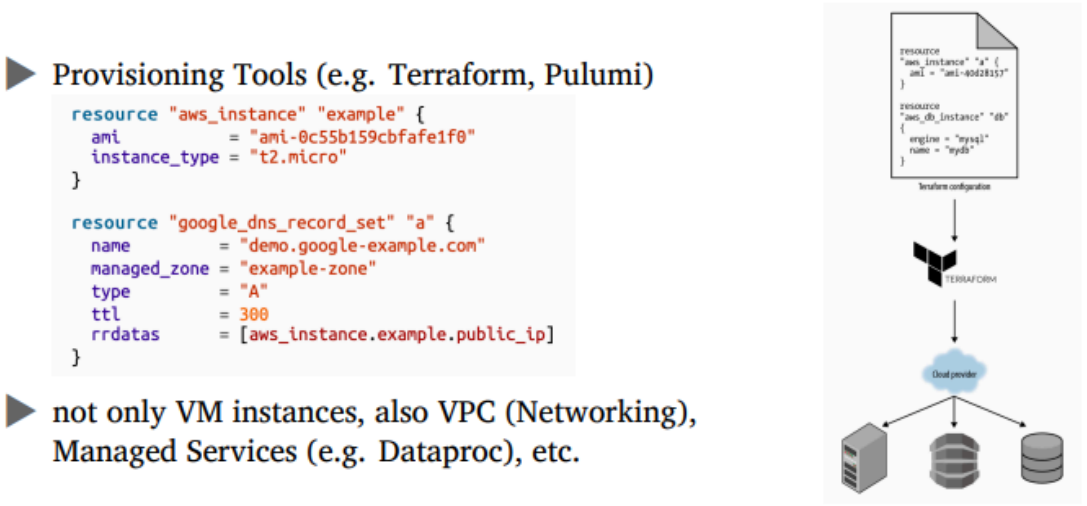
- narzędzia zarządzające konfiguracją:



- narzędzia ze schematami serwerów:



- narzędzia z providerami (*provisioningu*):



Terraform

- narzędzie tworzące architekturę
- przystosowane do chmury
- deklaratywny język programowania HCL

Podstawowe komendy

- `init` - przygotowanie katalogu roboczego na inne komendy
- `validate` - sprawdź czy konfiguracja jest poprawna
- `plan` - pokaż zmiany wymagane przez obecną konfigurację, generuje plan działania i wizualizuje zmiany
- `apply` - stwórz lub zmodyfikuj infrastrukturę (wykonaj zmiany pokazane przez `plan`)
- `destroy` - usuwa **tylko i wyłącznie** obiekty aktualnie zarządzane przez terraform i zapisane w pliku `terraform.tfstate`

Dostawcy (*providers*)

- zawierają typy **zasobów** oraz **danych**, dzięki którym można komunikować zmiany do zdalnych API
- posiadają drobne lokalne narzędzia, np. do generowania losowych liczb do unikalnego nazywania zasobów

```
provider "google" {
  project = var.project_name
  region  = var.region
}
terraform {
  required_providers {
    google = {
      version = "~> 4.8.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "3.1.2"
    }
    kubectl = {
      source  = "gavinbunney/kubectl"
      version = "1.14.0"
    }
  }
}
```

Zasoby (*Resources*)

- każdy zasób ma wejścia (argumenty) i wyjścia (atrybuty)
- do atrybutów (wyjść) zasobów można się odwoływać w innych zasobach
- wyliczone atrybuty są dostępne tylko po utworzeniu zasobu (np. URL albo ID)

Źródła danych

- reprezentują informacje *read-only* pobierane od dostawców

Zmienne

- wejściowe i wyjściowe
- przesyłane jako zmienne źrodowiskowe lub w pliku `.tfvars`
- mogą być lokalne, tylko dla konkretnych modułów
- mogą mieć walidację
- można definiować `locals` do wielokrotnego użycia wyrażeń

Zależności

- niejawne:
  - wynikają z odwołań do atrybutów innych zasobów
- jawne:
  - `depends_on` wymusza kolejność tworzenia lub usuwania

Zarządzanie stanami

- lokalne (domyślnie)
  - plik `terraform.tfstate` w formacie JSON
  - niebezpieczne i podane na błędy
- pamięć współdzielona
  - wymaga zdalnego backendu, np. GCS
  - szyfrowana komunikacja
  - wersjonowanie
  - współpraca z zespołem

Moduły

- każdy zestaw plików konfiguracyjnych Terraform w katalogu jest modułem

- zawsze mamy przynajmniej moduł `root`
- reużywalność kodu
- można go przechowywać w gicie
- stosy
  - kolekcje modułów które są logicznie powiązane ze sobą
  - reprezentują pojedynczą jednostkę wdrożenia infrastruktury, np. środowisko, lub jego część (system plików)
  - ograniczają zakres zmian w zasobach
  - szybsze niż zarządzanie stanami wszystkich zasobów w jednym pliku
  - rozdzielenie odpowiedzialności

Pętle i wyrażenia

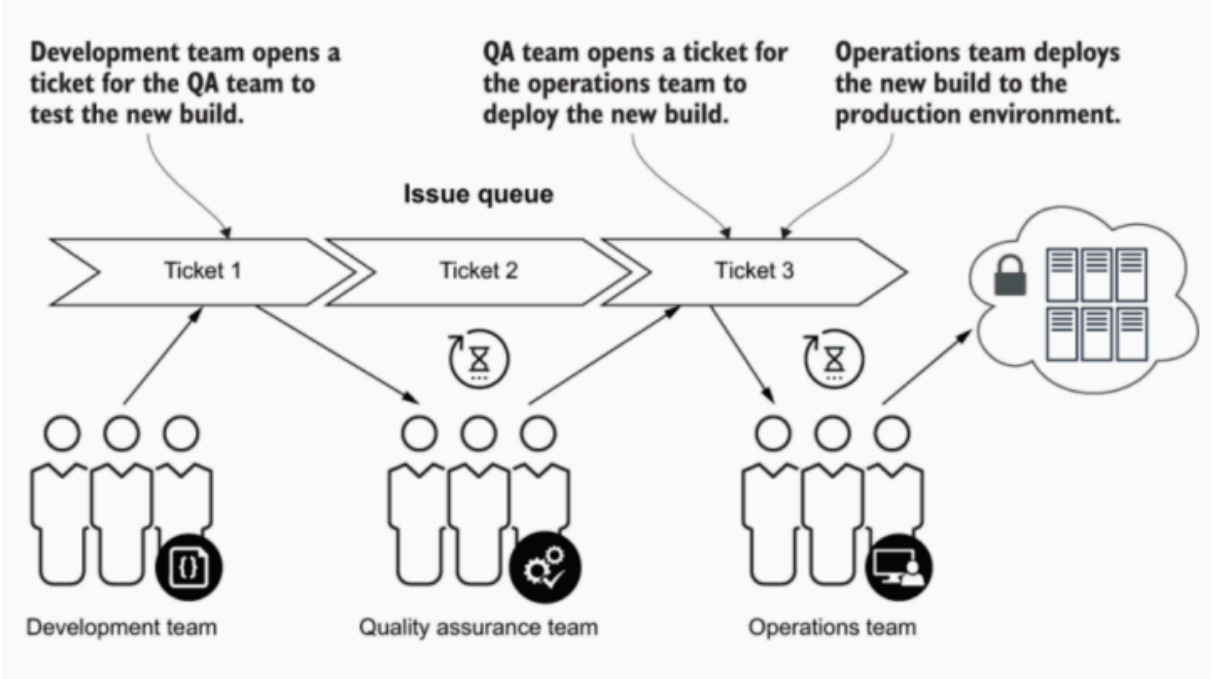
- `count`
  - przydatny gdy chcemy utworzyć wiele podobnych zasobów
- `for_each`
  - wygodny gdy każdy zasób identyfikujemy kluczem lub nazwą
- wyrażenia warunkowe
  - przydatne do opcjonalnych modułów i parametrów

Terragrunt

- orkiestrator
- upraszcza reużywalność kodu IaC i definicję środowisk
- każda jednostka może mieć własny stan i własny plik `terragrunt.hcl`

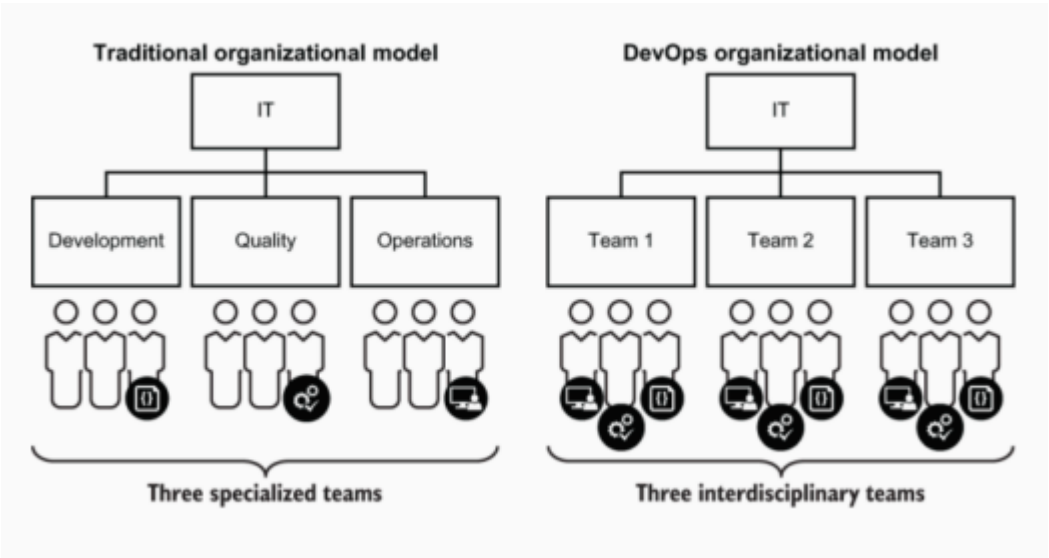
CloudOps

Klasyczne Ops



- rozdzielenie:
  - software developmentu (Dev)
  - zapewnienia jakości (QA)
  - konfiguracji infrastruktury (IT Ops)
  - wdrażania oprogramowania (IT Ops)
- powstają silosy, między którymi komunikacja odbywa się poprzez proces biletowy (*tickets*)

Dev(Sec)Ops



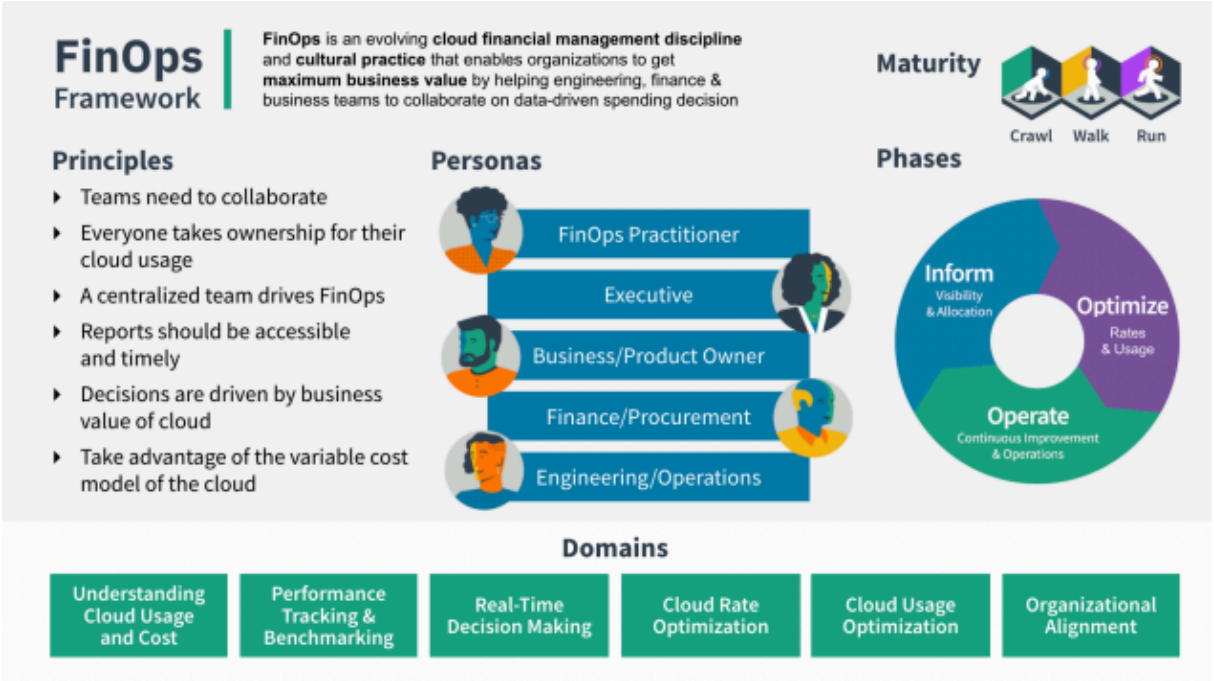
- praktyki łączące rozwój oprogramowania (Dev) z operacjami IT (Ops)
- skrócenie cyklu rozwoju oprogramowania
- skupienie na kliencie
- nacisk na automatyzację (CI/CD)

- częste dostarczanie funkcji
- zespoły stają się interdyscyplinarne i biorą na siebie pełną odpowiedzialność *end-to-end*

CloudOps

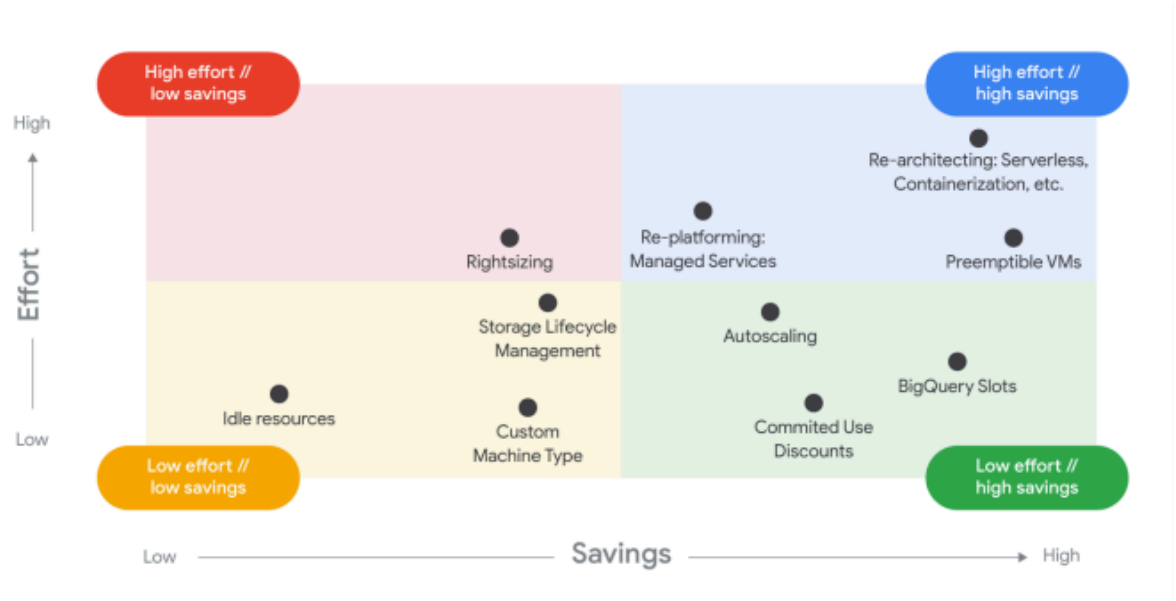
- DevOps dla chmury obliczeniowej
- IaC i automatyzacja
- monitorowanie i logowanie
- częste wyzwania:
  - środowiska multi-cloud
  - większa złożoność usług zarządzanych
  - potrzeba kontroli kosztów
- powody powstania
  - brak dostępnych umiejętności operacji chmurowych
  - rosnąca złożoność systemów
  - większe zagrożenia

FinOps



- praktyka i dyscyplina finansowego zarządznia chmurą
- celem jest uzyskanie maksymalnej wartości biznesowej
- 3 kluczowe fazy:
  - **inform** - widoczność kosztów
  - **optimize** - optymalizacja stawek i zużycia
  - **operate** - ciągłe doskonalenie

Taktyki oszczędzania



- etykietowanie zasobów dla rozliczania kosztów między zespołami i środowiskami
- budżety i alerty kosztowe
- dobór właściwych rozmiarów instancji (*rightsizing*)
- harmonogramy wyłączania nieużywanych zasobów
- autoscaling
- wybór tańszych klas storage lub modeli zakupu

XOps

GitOps

- proces DevOps, gdzie Git jest jedynym źródłem prawdy dla deklaratywnego kodu infrastruktury i aplikacji
- zmiany w kodzie automatycznie wyzwalają potoki CI/CD, aby środowisko produkcyjne odzwierciedlało kod
- obejmuje nie tylko kod aplikacji, ale też konfigurację i monitoring

DataOps

- zastosowanie najlepszych praktyk zwinnych metodyk i DevOps do przetwarzania danych
- automatyczne testowanie jakości danych
- automatyczne generowanie metadanych
- lineage danych
- tagowanie i polityki oparte o tagi

MLOps

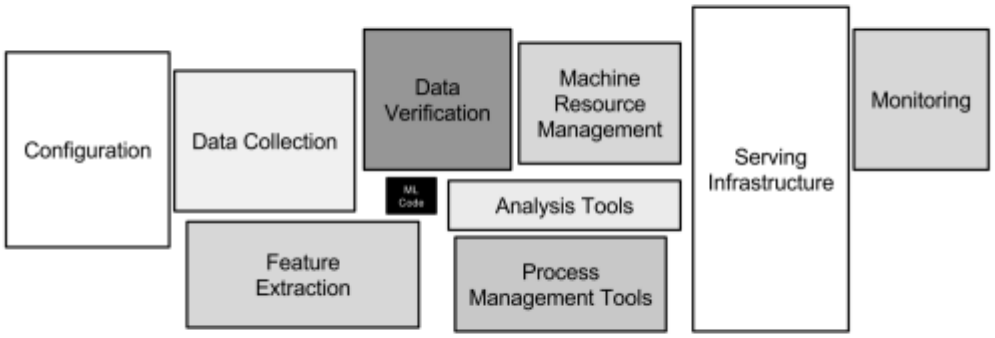


Figure: Elements of ML systems

- to samo ale do *Machine-Learingu*
- składa się z:
  - DataOps (np. Delta Lake z wersjonowaniem danych)
  - DevOps (wersjonowanie kodu z Gitem)
  - ModelOps (zarządzanie cyklem życia modelu)

AIOps

- *Agentic AI Ops*, albo *Artificial Intelligence for IT Operations*
- łączenie AI i uczenia maszynowego do automatyzacji i poprawy operacji IT
- lepsze monitorowanie, debugowanie i odpowiedzi na incydenty w skomplikowanych środowiskach

Automatyzacje

- IaC - użycie Terraform, Terragrunt czy Atlantis do zarządzania zasobami
- Git Hooks - np. pre-commit - framework do automatycznego sprawdzania błędów przed i po commitach
- GitHub Actions - automatyzacja workflows bezpośrednio w repo (np. bezpieczne uwierzytelnienie bezkluczowe)
- SecOps - integracja narzędzi automatycznie sprawdzających bezpieczeństwo (np. weryfikacja konfiguracji IaC)
- keyless authentication
  - brak długowiecznych kluczy JSON
  - krótkotrwałe tokeny rotowane przy wdrożeniach
- FinOps automation
  - np. Infracost do estymacji kosztów zmian Terraform w pull requestach